

UNIT 3: JOIN

What is a JOIN?

A **JOIN** is used to combine rows from **two or more tables** based on a related column between them.

Why do we use JOIN?

Suppose we have two tables:

Student Table

Student_ID	Name	Dept_ID
------------	------	---------

101	Amit	1
-----	------	---

102	Riya	2
-----	------	---

103	Karan	1
-----	-------	---

Department Table

Dept_ID	Department
---------	------------

1	CSE
---	-----

2	IT
---	----

Instead of storing department name repeatedly in Student table, we store Dept_ID.

To display complete information, we use JOIN.

Types of JOIN

There are mainly **5 types** of joins.

JOIN

|

├── INNER JOIN

├── LEFT JOIN

├── RIGHT JOIN

├── FULL OUTER JOIN

└── SELF JOIN

1. INNER JOIN

Definition

Returns **only matching records** from both tables.

Syntax

MySQL

SELECT columns

FROM Student

INNER JOIN Department

ON Student.Dept_ID = Department.Dept_ID;

SQL*Plus (Oracle)

SELECT s.Student_ID,

s.Name,

d.Department

FROM Student s

INNER JOIN Department d

ON s.Dept_ID=d.Dept_ID;

Example

Student

ID	Name	Dept_ID
----	------	---------

101	Amit	1
-----	------	---

102	Riya	2
-----	------	---

103	Karan	3
-----	-------	---

Department

Dept_ID	Department
---------	------------

1	CSE
---	-----

2	IT
---	----

Query

SELECT Name, Department

FROM Student

INNER JOIN Department

ON Student.Dept_ID=Department.Dept_ID;

Output

Name Department

Amit CSE

Riya IT

Karan is not displayed because Dept_ID=3 has no match.

2. LEFT JOIN (LEFT OUTER JOIN)

Definition

Returns

- All records from left table
- Matching records from right table
- If no match → NULL

Syntax

MySQL

SELECT *

FROM Student

LEFT JOIN Department

ON Student.Dept_ID=Department.Dept_ID;

SQL*Plus

SELECT *

FROM Student s

LEFT JOIN Department d

ON s.Dept_ID=d.Dept_ID;

Output

Name Department

Amit CSE

Riya IT

Karan NULL

3. RIGHT JOIN (RIGHT OUTER JOIN)

Definition

Returns

- All rows from right table
- Matching rows from left table

MySQL

SELECT *

FROM Student

RIGHT JOIN Department

ON Student.Dept_ID=Department.Dept_ID;

SQL*Plus (Oracle)

Modern Oracle supports:

SELECT *

FROM Student s

RIGHT JOIN Department d

ON s.Dept_ID=d.Dept_ID;

Older Oracle SQL*Plus syntax:

SELECT *

FROM Student s,

Department d

WHERE s.Dept_ID(+)=d.Dept_ID;

4. FULL OUTER JOIN

Definition

Returns

- All rows from left table
- All rows from right table

Matched rows are merged.

SQL*Plus (Oracle)

```
SELECT *
```

```
FROM Student s
```

```
FULL OUTER JOIN Department d
```

```
ON s.Dept_ID=d.Dept_ID;
```

MySQL

MySQL **does not support FULL OUTER JOIN directly.**

Alternative

```
SELECT *
```

```
FROM Student
```

```
LEFT JOIN Department
```

```
ON Student.Dept_ID=Department.Dept_ID
```

```
UNION
```

```
SELECT *
```

```
FROM Student
```

```
RIGHT JOIN Department
```

```
ON Student.Dept_ID=Department.Dept_ID;
```

5. SELF JOIN

Definition

A table joins with itself.

Used when data inside one table is related.

Example

Employee Table

Emp_ID	Name	Manager_ID
--------	------	------------

1	Raj	NULL
---	-----	------

2	Amit	1
---	------	---

3	Riya	1
---	------	---

Query

```
SELECT E.Name Employee,
```

```
       M.Name Manager
```

```
FROM Employee E
```

```
JOIN Employee M
```

```
ON E.Manager_ID=M.Emp_ID;
```

Output

Employee	Manager
----------	---------

Amit	Raj
------	-----

Riya	Raj
------	-----

Built-in Functions

What are Built-in Functions?

Functions provided by DBMS to perform calculations, manipulate strings, dates, and aggregate values.

Types of Built-in Functions

Built-in Functions

|

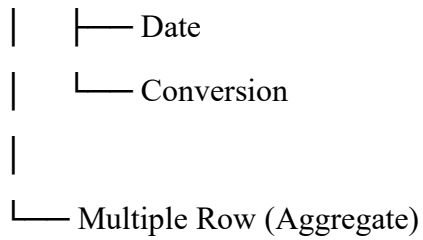
|— Single Row Functions

|

|— Character

|

|— Numeric



1. Character Functions

Operate on text.

Function	Description
UPPER()	Uppercase
LOWER()	Lowercase
LENGTH()	Find length
CONCAT()	Join strings
SUBSTRING()/SUBSTR()	Extract text
TRIM()	Remove spaces

Example

MySQL

```
SELECT UPPER('hello');
```

Output

HELLO

SQL*Plus

```
SELECT UPPER('hello')
```

```
FROM dual;
```

LENGTH

MySQL

```
SELECT LENGTH('Database');
```

SQL*Plus

```
SELECT LENGTH('Database')
```

FROM dual;

Output

8

CONCAT

MySQL

```
SELECT CONCAT('Hello',' Student');
```

SQL*Plus

```
SELECT 'Hello' || ' Student'
```

FROM dual;

2. Numeric Functions

Function	Purpose
ROUND()	Round value
CEIL()/CEILING()	Upper integer
FLOOR()	Lower integer
MOD()	Remainder
ABS()	Absolute value
POWER()	Power

Example

```
SELECT ROUND(12.456,2);
```

Output

12.46

3. Date Functions

MySQL

```
SELECT CURDATE();
```

```
SELECT NOW();
```

SQL*Plus

```
SELECT SYSDATE
```

```
FROM dual;
```

Add days

MySQL

```
SELECT DATE_ADD(CURDATE(),INTERVAL 5 DAY);
```

SQL*Plus

```
SELECT SYSDATE+5
```

```
FROM dual;
```

4. Conversion Functions

Convert one datatype to another.

SQL*Plus

```
SELECT TO_CHAR(SYSDATE,'DD-MM-YYYY')
```

```
FROM dual;
```

```
SELECT TO_DATE('25-05-2025','DD-MM-YYYY')
```

```
FROM dual;
```

MySQL

```
SELECT CAST(100 AS CHAR);
```

5. Aggregate Functions

Used on multiple rows.

Function Purpose

COUNT() Count

SUM() Total

AVG() Average

MAX() Maximum

Function Purpose

MIN() Minimum

Example

Employee

Salary

30000

45000

50000

SELECT AVG(Salary)

FROM Employee;

Output

41666.67

Other Database Objects

Database Objects

Database Objects

|

|— View

|— Synonym

|— Index

VIEW

Definition

A **View** is a **virtual table** created from one or more tables.

It stores only the SQL query, not the actual data.

Advantages

- Security
- Simplicity

- Hide sensitive columns
- Reuse complex queries

Syntax

MySQL

```
CREATE VIEW Student_View AS
```

```
SELECT Student_ID,
```

```
    Name
```

```
FROM Student;
```

SQL*Plus

```
CREATE VIEW Student_View AS
```

```
SELECT Student_ID,
```

```
    Name
```

```
FROM Student;
```

```
Display
```

```
SELECT *
```

```
FROM Student_View;
```

```
Delete
```

```
DROP VIEW Student_View;
```

SYNONYM

Definition

A **Synonym** is an **alternate name (alias)** for a database object such as a table, view, sequence, or procedure.

Advantages

- Simplifies object names.
- Allows users to reference objects without knowing the owner's schema.
- Helps when an object is moved or renamed (the synonym can remain the same).

SQL*Plus (Oracle)

Create

```
CREATE SYNONYM Emp FOR Employee;
```

Use

```
SELECT *
```

```
FROM Emp;
```

Drop

```
DROP SYNONYM Emp;
```

MySQL

MySQL does not support SYNONYM objects. A common alternative is to create a VIEW:

```
CREATE VIEW Emp AS
```

```
SELECT *
```

```
FROM Employee;
```

INDEX

Definition

An **Index** increases the speed of searching data.

Think of it like the index in a book.

Without index → Full table scan

With index → Direct search

Advantages

- Faster SELECT
- Faster searching
- Better performance

Disadvantages

- Uses extra storage
- Slower INSERT, UPDATE, DELETE because indexes must also be maintained

Types of Index

1. Single Column Index

```
CREATE INDEX idx_name  
ON Student(Name);
```

2. Composite Index

```
CREATE INDEX idx_student  
ON Student(Name,Dept_ID);
```

3. Unique Index

```
CREATE UNIQUE INDEX idx_email  
ON Student(Email);
```

Delete Index

MySQL

```
DROP INDEX idx_name  
ON Student;
```

SQL*Plus

```
DROP INDEX idx_name;
```

Transaction Control Statements (TCL)

What is Transaction?

A **transaction** is a logical unit of work consisting of one or more SQL statements that should be completed together.

Example:

Transfer ₹1000

Debit Account A

↓

Credit Account B

↓

Commit

If one step fails, all changes should be undone.

Transaction Control Statements

TCL

|

|— COMMIT

|— ROLLBACK

|— SAVEPOINT

COMMIT

Definition

Makes all changes **permanent**.

Syntax

COMMIT;

Example

UPDATE Employee

SET Salary=60000

WHERE Emp_ID=101;

COMMIT;

After COMMIT, the changes cannot be undone using ROLLBACK.

ROLLBACK

Definition

Cancels all changes made after the last COMMIT.

Syntax

```
ROLLBACK;
```

Example

```
DELETE FROM Employee  
WHERE Emp_ID=105;
```

```
ROLLBACK;
```

The deleted row is restored because the transaction was rolled back before committing.

SAVEPOINT

Definition

Creates a checkpoint within a transaction so you can roll back to a specific point instead of undoing the entire transaction.

Syntax

```
SAVEPOINT savepoint_name;
```

Example

```
UPDATE Employee  
SET Salary=50000  
WHERE Emp_ID=101;  
SAVEPOINT sp1;  
UPDATE Employee  
SET Salary=60000  
WHERE Emp_ID=102;  
ROLLBACK TO sp1;  
COMMIT;
```

Explanation

1. Salary of Employee 101 is updated.
2. SAVEPOINT sp1 is created.

3. Salary of Employee 102 is updated.
4. ROLLBACK TO sp1 undoes only the second update.
5. COMMIT permanently saves the first update.

Difference Between COMMIT, ROLLBACK, and SAVEPOINT

Feature	COMMIT	ROLLBACK	SAVEPOINT
Purpose	Save changes permanently	Undo changes	Create a checkpoint
Effect	Makes changes permanent	Restores previous state	Marks a position in the transaction
Can changes be recovered?	No	Yes (before COMMIT)	Yes, by rolling back to the savepoint
Syntax	COMMIT;	ROLLBACK;	SAVEPOINT sp1;

Interview & Viva Questions

1. What is a JOIN? Why is it used?
2. Explain the difference between INNER JOIN and LEFT JOIN.
3. What is a SELF JOIN? Give a real-world example.
4. Why doesn't MySQL support FULL OUTER JOIN directly?
5. What are built-in functions? Classify them.
6. What is the difference between single-row and aggregate functions?
7. What is a View? How is it different from a table?
8. What is a Synonym? Does MySQL support synonyms?
9. What is an Index? What are its advantages and disadvantages?
10. What is a transaction in DBMS?
11. Explain COMMIT, ROLLBACK, and SAVEPOINT with examples.
12. What happens if you execute ROLLBACK after COMMIT?